Efficient LLM Serving with KV Cache, Speculative Decoding, and Quantization

Abstract

The deployment of Large Language Models (LLMs) has transitioned from a research curiosity to a critical infrastructure component, necessitating rigorous optimization of inference serving systems. As model scales expand and context windows lengthen, the traditional bottlenecks of memory bandwidth, latency, and computational cost have become acute. This survey provides a comprehensive analysis of three pivotal pillars of efficient LLM serving: Key-Value (KV) cache management, speculative decoding, and quantization. We synthesize findings from 300 recent academic works to construct a unified taxonomy of optimization strategies. We examine how KV cache compression, offloading, and sharing mechanisms address the memory capacity constraints inherent in long-context generation. We detail the evolution of speculative decoding from simple draft-verify paradigms to complex tree-based, adaptive, and multi-model collaborative frameworks that overcome sequential generation limits. Furthermore, we explore advanced quantization techniques, ranging from post-training weight compression to mixed-precision KV cache strategies and hardware-software co-design. By integrating insights across algorithmic innovations, system architectures, and hardware accelerators, this paper identifies current limitations, evaluates trade-offs between accuracy and efficiency, and outlines future research directions for scalable, low-latency LLM inference.

1. Introduction

The rapid proliferation of Large Language Models (LLMs) has fundamentally altered the landscape of artificial intelligence, enabling capabilities in reasoning, code generation, and creative writing that were previously unattainable. However, the deployment of these models at scale presents formidable challenges. The autoregressive nature of transformer-based generation imposes a strict sequential dependency, where each token generation requires a full forward pass, leading to significant latency. Moreover, the memory footprint of the Key-Value (KV) cache grows linearly with sequence length, creating a "memory wall" that limits batch sizes and context windows [1], [2], [3]. As models evolve from billions to hundreds of billions of parameters, and context windows extend to millions of tokens, the inefficiencies of naive inference become prohibitive for real-time applications [4], [5].

To address these challenges, the research community has converged on three primary axes of optimization: KV cache management, speculative decoding, and quantization. KV cache optimization seeks to mitigate memory bottlenecks through compression, eviction, and hierarchical storage, allowing for longer contexts and higher concurrency [6], [7], [8]. Speculative decoding breaks the sequential barrier by generating multiple candidate tokens in parallel using a smaller "drafter" model or internal mechanisms, verifying them simultaneously with the target model to achieve lossless acceleration [9], [10], [11]. Quantization reduces the precision of weights and activations, decreasing memory usage and increasing computational throughput, though often at the cost of potential accuracy degradation which modern techniques strive to minimize [12], [13], [14].

Recent literature reveals a shift from isolated optimizations to holistic system co-design. For instance, speculative decoding strategies are increasingly integrated with KV cache compression to handle long contexts where memory bandwidth dominates latency [15], [16]. Similarly, quantization is no longer limited to static post-training methods but involves dynamic, layer-wise, and token-adaptive precision selection [17], [14]. This survey synthesizes 300 relevant papers to provide a deep technical overview of these intersecting fields. We categorize methodologies into coherent

families, analyzing their theoretical underpinnings, implementation details, and empirical performance. By examining the synergy between algorithmic innovations and system-level engineering, we aim to provide a roadmap for developing next-generation LLM serving infrastructures that are both efficient and scalable.

2. Method

This section organizes the vast landscape of efficient LLM serving into three primary methodological domains: KV Cache Management, Speculative Decoding, and Quantization. Within each domain, we further delineate specific technical approaches, ranging from algorithmic modifications to system-level architectural changes.

2.1 KV Cache Management and Optimization

The KV cache, which stores intermediate attention states to avoid recomputation, is the primary consumer of GPU memory during the decoding phase. Efficient management of this cache is critical for supporting long contexts and high concurrency.

2.1.1 KV Cache Compression and Quantization A dominant strategy for reducing KV cache footprint is compression via quantization and low-rank approximation. **Oaken** introduces an online-offline hybrid quantization pipeline that separates KV values into groups based on offline-determined thresholds, applying a group-shift algorithm to compress outliers into low bitwidths, achieving significant throughput speedups with minimal accuracy loss [6]. Similarly, **VecInfer** employs dual smooth and Hadamard transformations to suppress outliers in the key cache before applying vector quantization, enabling comprehensive codebook coverage and fused dequantization kernels [12]. **GEAR** proposes a synergistic approach combining ultra-low precision quantization, low-rank approximation for coherent residuals, and sparse matrices for outliers, demonstrating that quantization errors contain recoverable structures [13].

Advanced compression techniques also leverage transform coding. **kvtc** applies PCA-based feature decorrelation using a global orthonormal basis, followed by dynamic programming for adaptive bit allocation, achieving high compression ratios with negligible accuracy drops [18]. **H1B-KV** pushes the limits further by projecting key vectors into 1-bit binary sketches via Gaussian random matrices, enabling bitwise attention operations while quantizing values to 4-bit integers [19]. **Coupled Quantization (CQ)** exploits inter-channel mutual information by grouping contiguous channels and jointly quantizing them to shared centroids, significantly reducing joint entropy compared to independent scalar quantization [20].

2.1.2 Hierarchical Storage and Offloading When the KV cache exceeds GPU High Bandwidth Memory (HBM), hierarchical storage and offloading become necessary. **OrbitFlow** utilizes a lightweight Integer Linear Programming (ILP) solver to determine optimal per-request KV cache layer placement between GPU and host memory, dynamically reconfiguring offload distances based on real-time SLO constraints [4]. **LayerKV** implements fine-grained layer-wise offloading during prefill, decoupling GPU memory constraints from prompt length and utilizing an SLO-aware scheduler to balance Time-to-First-Token (TTFT) and Time-Per-Output-Token (TPOT) [21].

System-level innovations also address I/O bottlenecks. **Tutti** eliminates CPU intervention via a GPU-centric object store using NVMe SSDs, employing GPU-native `io_uring` for asynchronous direct access, thereby saturating NVMe bandwidth [22]. **Strata** decouples GPU and CPU memory

layouts using GPU-assisted I/O kernels to transfer fragmented KV pages efficiently, avoiding the trade-off between page size and bandwidth efficiency [23]. **MemServe** introduces an elastic memory pool abstracting heterogeneous memory (CPU DRAM and GPU HBM) to unify context caching and disaggregated inference, enabling seamless KV cache persistence across requests [24]. **FastSwitch** optimizes context switching by merging contiguous blocks to reduce API dispatch overhead and employing asynchronous multithreaded swap managers to overlap I/O with computation [25].

2.1.3 KV Cache Sharing and Reuse In multi-turn and multi-agent scenarios, significant redundancy exists in KV caches. **PRISM** integrates a Query-Aware Scheduler with a Demand-Aware Radix Tree to group requests by shared prefixes, aligning admission control with cache residency to reduce effective prefill costs [26]. **CacheBlend** accelerates Retrieval-Augmented Generation (RAG) by fusing pre-computed KV caches of multiple text chunks, selectively recomputing only High-KV-Deviation (HKVD) tokens identified via gradual filtering [27]. **SparseX** utilizes contiguous token segments as reuse units with RoPE alignment, employing Sparse-Q indices to estimate key tokens requiring correction, thus enabling efficient sharing for interleaved contexts [28].

For multi-agent systems, **Tokencake** implements event-driven proactive KV cache offloading triggered by function call events, eliminating idle GPU occupancy during predictable tool execution intervals [29]. **TokenDance** scales multi-agent serving by implementing round-aware collective processing, amortizing RoPE rotation and diff analysis costs across all agents in a round [30]. **ForkKV** addresses Multi-LoRA serving by decoupling the KV cache into a shared base cache and lightweight residual caches using a DualRadixTree architecture with copy-on-write semantics [31]. **DroidSpeak** enables cross-LLM communication by identifying critical layer groups sensitive to KV deviations and selectively recomputing them while reusing the rest of the cache across fine-tuned variants [32].

2.1.4 Dynamic Eviction and Sparsity Dynamic eviction policies manage cache size by discarding less important tokens. **Lethe** implements dual-dimensional pruning, allocating token budgets per layer based on runtime sparsity estimators and employing Recency-Aware Selective Retention (RASR) to handle temporal fluctuations in attention patterns [33]. **A2SF** introduces a forgetting factor into accumulative attention scores to neutralize causal mask bias, ensuring recent tokens are prioritized without ignoring critical historical context [34]. **Keyformer** reduces cache size by retaining a fixed budget of recent tokens and selecting critical "key" tokens via a score function regularized with Gumbel noise to counteract softmax disruption [35].

Sparse attention mechanisms further reduce compute and memory loads. **LServe** unifies static streaming heads and dynamic page-level sparsity into a single block-sparse framework, employing hierarchical paging to separate logical importance from physical storage [16]. **Progressive Sparse Attention (PSA)** replaces fixed top-k selection with threshold-based progressive accumulation, halting execution once accumulated attention weights exceed a confidence threshold [36]. **TidalDecode** implements Position Persistent Sparse Attention, executing full attention in initial layers to identify top-k tokens and reusing this set for subsequent layers, leveraging the spatial coherence of attention scores [37]. **EntmaxKV** formulates sparse decoding as exact support recovery for alpha-entmax attention, selecting candidate pages likely containing the non-zero support set via lightweight metadata scoring [38].

2.2 Speculative Decoding and Parallel Generation

Speculative decoding (SD) accelerates inference by generating candidate tokens with a faster mechanism and verifying them in parallel with the target model, effectively breaking the autoregressive bottleneck.

2.2.1 Draft-then-Verify Paradigms The foundational approach involves a small "drafter" model generating tokens for parallel verification. **Speculative Sampling (SpS)** generates K candidates via a fast draft model and verifies them using modified rejection sampling to recover the exact target distribution [9]. **Fast Inference from Transformers** generalizes this to stochastic sampling, employing a small approximation model to draft tokens and verifying them in a single forward pass of the target model [10]. To improve drafter quality, **Recurrent Drafter (ReDrafter)** employs a lightweight RNN conditioned on the target LLM's hidden states, capturing local dependencies better than independent heads [39]. **Chimera** integrates a trigram encoder with residual decoding heads that fuse draft and original LLM hidden states, enabling the drafter to access full context information [40].

Advanced drafting strategies include **Minions**, which aggregates outputs from multiple Small Speculative Models (SSMs) via tree-based weighted voting to maximize acceptance rates without increasing verification load [41]. **SPIN** utilizes a learning-based SSM selector modeled as a multi-armed bandit to assign optimal heterogeneous drafters to requests based on difficulty [42]. **SpecRouter** formulates inference as adaptive routing across a heterogeneous model pool, dynamically constructing multi-level inference paths to minimize effective latency [43]. **Jakiro** boosts SD by using Mixture of Experts (MoE) heads to dynamically decouple intra-layer token predictions, enhancing candidate diversity [44].

2.2.2 Tree-Based and Multi-Candidate Verification To increase the number of accepted tokens per step, recent works organize candidates into trees. **SpecInfer** constructs token trees via expansion or merge methods and employs tree-based parallel decoding to verify all candidates in a single LLM pass using topology-aware causal masks [45]. **OPT-Tree** dynamically constructs optimal draft tree structures per decoding step via greedy search to maximize the mathematical expectation of acceptance length [46]. **Multi-Candidate Speculative Decoding (MCSD)** samples k candidates per position and organizes them for parallel verification, maintaining the target distribution through a novel sampling algorithm [47]. **Lookahead** builds a dynamic trie tree storing n-gram tokens from prompts and generated responses, retrieving hierarchical multi-branch drafts to maximize token density within the Critical Decoding Length [48].

2.2.3 Self-Speculative and Internal Mechanisms Eliminating the need for external draft models, self-speculative methods leverage the target model's own redundancy. **Draft & Verify** utilizes a single LLM, selectively skipping intermediate layers during the drafting stage to generate tokens rapidly, with Bayesian optimization determining the optimal skip pattern [49]. **SPACE** integrates Semi-Autoregressive Supervised Fine-Tuning (SAR-SFT) to train the model to predict multiple tokens via masked objectives, enabling efficient speculative generation without auxiliary models [50]. **BiTA** injects learnable prompt and mask tokens into frozen autoregressive models to enable semi-autoregressive generation, flattening draft candidate trees for single forward pass verification [51]. **Cassandra** constructs a draft model via unstructured value pruning and mantissa truncation on the target weights, maintaining high acceptance rates without extra memory overhead [52].

2.2.4 Adaptive and Confidence-Modulated Decoding Static speculation lengths are often suboptimal. **Nightjar** integrates a Contextual Multi-Armed Bandit planner to select optimal speculative lengths per step based on real-time batch size, explicitly penalizing switching costs [53]. **Confidence-Modulated Speculative Decoding (CM-ASD)** uses a lightweight confidence estimator computing entropy or logit margins to dynamically modulate drafting length [54]. **AdaSD** implements dual adaptive thresholds updated in real-time, halting draft generation when token entropy exceeds a running mean and adjusting verification thresholds based on Jensen-Shannon distance [55]. **AdaEDL** approximates a lower bound on token acceptance probability using draft model logits entropy to trigger early stopping, preventing wasted computation on low-probability continuations [56]. **SpecServe** employs a predict-execute-correct strategy where a confidence prior verifier eliminates low-probability tokens to enable request-level control [57].

2.2.5 Speculative Decoding for Specialized Domains Speculative decoding is increasingly adapted for multimodal and reasoning tasks. **Multimodal Speculative Decoding (MSD)** decouples text and visual token processing, bypassing image encoder overhead for draft models by feeding raw embeddings directly [58]. **Sparrow** employs Hidden State Reuse to inject target model's deep text-hidden states containing visual semantics into the draft model, eliminating visual KV cache overhead during drafting for Video-LLMs [59]. **Speculative Search (SpecSearch)** employs a bi-level speculative thought generator for reasoning tasks, using a small model for coarse-grained thought drafting and a large model for fine-grained token correction, guided by a Process Reward Model [60]. **Nevermind** utilizes speculative decoding streams as a proxy for pre-output thought processes to apply inhibition factors for safety moderation, suppressing unsafe token probabilities in parallel [61].

2.3 Quantization and Model Compression

Quantization reduces the numerical precision of model parameters and activations to decrease memory footprint and increase computational efficiency.

2.3.1 Weight and Activation Quantization Post-training quantization (PTQ) is widely adopted for its ease of deployment. **SmoothQuant+** applies channel-wise smoothing to migrate activation outliers to weights, enabling seamless 4-bit weight quantization with minimal accuracy loss [62]. **Atom** employs mixed-precision quantization, retaining outliers in INT8 while compressing normal values to INT4 via channel reordering, fused with custom GEMM kernels [63]. **FlexQ** implements fine-grained group quantization for weights and activations, utilizing a bit-level decomposition engine to emulate INT6 matrix multiplication on Binary Tensor Cores [64]. **Any-Precision LLM** generates a seed model at minimum bit-width and incrementally upscales to higher bit-widths by appending bits, preserving model quality across a range of precisions without retraining [65].

For extreme compression, **BTC-LLM** integrates learnable transformations to redistribute activation outliers and employs binary codebooks to compress binary weight vectors, achieving sub-1-bit quantization [66]. **TerEffic** implements 1.6-bit ternary weight compression on FPGAs, encoding five ternary weights into 8 bits and utilizing LUT-based cores to eliminate DSP dependency [67]. **BlockDialect** assigns optimal FP4 variants from a 16-dialect formatbook to each fine-grained block based on local data distribution, outperforming uniform scaling methods [68].

2.3.2 Mixed-Precision and Layer-Wise Strategies Recognizing that not all layers or tokens require the same precision, mixed-precision strategies offer a better accuracy-efficiency trade-off. **KVTuner** formulates layer-wise KV precision pair selection as a discrete combinatorial optimization problem, employing two-stage search space pruning to minimize memory usage while maintaining accuracy [14]. **MorphServe** implements a closed-loop control system executing asynchronous layer swapping to replace low-sensitivity full-precision layers with quantized versions at runtime [17]. **Cocktail** segments context into chunks and assigns bitwidths (FP16, INT4, INT2) based on RAG-encoded similarity to the query, allowing aggressive quantization for irrelevant chunks [69]. **MoQAE** treats quantization bit-width configurations as experts within a Mixture of Experts framework, selecting optimal bit-widths per chunk via a trainable router [70].

2.3.3 Hardware-Software Co-Design Efficient quantization often requires specialized hardware support. **MixPE** integrates into a systolic array using an output-stationary dataflow, executing mixed-precision GEMM directly by replacing multipliers with shift-and-add operations for INT4 weights [71]. **LUT-LLM** replaces arithmetic linear layers with memory-based 2D table lookups on FPGAs using activation-weight co-quantization, leveraging abundant on-chip memory [72]. **Quasar** replaces the full-precision target model verifier in speculative decoding with a W8A8 quantized counterpart, utilizing enhanced SmoothQuant to preserve logit distribution fidelity [73]. **EVA** reformulates memory-bound GEMV decoding into compute-bound GEMM by computing dot products between input vectors and weight codebooks, enabling conflict-free lookups [74].

2.4 System-Level Architectures and Scheduling

Beyond algorithmic optimizations, system-level architectures play a crucial role in efficient serving.

2.4.1 Disaggregated Prefill-Decode Separating the compute-bound prefill phase from the memory-bound decode phase allows for specialized resource allocation. **Dej´ aVu** decouples prompt processing and token generation into separate machine pools, implementing microbatch-level KV cache swapping to eliminate pipeline bubbles [75]. **BanaServe** decouples resource allocation from state management via layer-level weight migration and a Global KV Cache Store, enabling purely load-aware routing [76]. **OmniInfer** decouples prefill and decode via a global scheduler, dynamically migrating MoE experts and employing evolutionary search for sparse attention patterns [77]. **ConServe** elevates scheduling granularity to the conversation level, enforcing a strict two-phase lifecycle with a dedicated high-throughput node for the first-turn prefill [78].

2.4.2 Scheduling and Resource Allocation Effective scheduling mitigates head-of-line blocking and optimizes resource utilization. **Sarathi-Serve** splits prefill requests into fixed-size compute chunks and coalesces them with ongoing decode iterations, exploiting the slack in memory-bound decode phases [79]. **ALISE** integrates a retrieval-based speculative scheduler estimating job execution time via BERT embeddings, employing priority queues for iteration-level preemptive scheduling [80]. **Sorted-F** minimizes total end-to-end latency by constructing batches that minimize a quality metric balancing batch size and output length, proving a constant competitive ratio [81]. **TokenFlow** integrates a buffer-aware scheduler with hierarchical KV cache management, dynamically prioritizing requests based on real-time token buffer occupancy to maximize effective throughput during bursts [82].

2.4.3 Distributed and Serverless Inference Distributed systems enable scaling beyond single-node limits. **PETALS** implements fault-tolerant pipeline parallelism over unreliable Internet con-

nections, utilizing dual attention caches for rapid recovery from node failures [83]. **ServerlessLLM** optimizes cold starts via loading-optimized checkpoint formats and token-based live migration, leveraging underutilized in-server multi-tier storage [84]. **GenTorrent** employs a decentralized overlay network with anonymous routing and distributed Hash-Radix trees for KV cache prefix matching, enabling peer-to-peer serving [85]. **TraCT** replaces RDMA network paths with direct GPU-CXL DMA for KV block transfer at rack-scale, utilizing a two-tier locking mechanism for mutual exclusion [86].

3. Challenges and Outlook

Despite significant advancements, several critical challenges remain in the quest for efficient LLM serving.

Accuracy-Efficiency Trade-offs: While quantization and pruning offer substantial memory savings, they often introduce accuracy degradation, particularly in complex reasoning tasks or low-bit regimes (<4-bit) [87], [88]. Activation outliers remain a persistent hurdle, requiring sophisticated smoothing or mixed-precision strategies that add implementation complexity [1], [89]. Future research must focus on robust, training-free quantization methods that can handle diverse domains without extensive calibration.

Dynamic Workload Adaptation: Most current systems rely on static configurations or offline profiling, which struggle with highly dynamic, bursty workloads typical of real-world deployments [53], [57]. Adaptive mechanisms that can reconfigure speculation lengths, quantization precisions, and cache eviction policies in real-time without incurring significant overhead are essential. The integration of reinforcement learning and online bandit algorithms shows promise but requires further validation in production environments [53], [42].

Hardware Heterogeneity: The diversity of hardware accelerators (GPUs, TPUs, FPGAs, NPUs) complicates the deployment of optimized kernels. Many advanced techniques, such as specific quantization formats (e.g., INT6, ternary) or sparse attention patterns, require custom kernel development that may not be portable across architectures [72], [71], [64]. There is a growing need for compiler-based approaches and intermediate representations that can automatically generate optimized kernels for diverse hardware targets [90], [91].

Security and Privacy: Speculative decoding and KV cache sharing introduce new attack surfaces. Side-channel attacks exploiting packet size variations in speculative decoding can leak sensitive prompt information [92]. Multi-tenant KV cache reuse raises concerns about data leakage between users, necessitating robust isolation mechanisms like selective prefix isolation [93]. Privacy-preserving split inference techniques that amortize network latency via speculation are emerging but often lack formal cryptographic guarantees [94].

Evaluation Bottlenecks: The field suffers from a lack of unified benchmarks and evaluation metrics. Performance is highly sensitive to workload characteristics (e.g., prompt length, output distribution), making fair comparison difficult [11], [95]. Many studies report speedups on specific datasets or hardware configurations that do not generalize. Comprehensive benchmarks covering diverse modalities, context lengths, and hardware platforms are urgently needed to drive reproducible research.

Future Directions: Future research should explore the convergence of these three pillars. For instance, integrating speculative decoding with quantized verification could yield compounding speedups [73]. Hierarchical KV cache management combined with semantic caching could dras-

tically reduce redundancy in agentic workflows [96]. Furthermore, the rise of non-transformer architectures (e.g., Mamba, RetNet) necessitates rethinking these optimization strategies for linear-time models [97], [98]. Finally, the development of "green AI" metrics that account for energy consumption and carbon footprint, rather than just latency and throughput, will be crucial for sustainable deployment [99], [100].

4. Conclusion

Efficient LLM serving is a multifaceted challenge requiring innovations across algorithms, systems, and hardware. This survey has synthesized a vast body of literature to highlight the transformative potential of KV cache management, speculative decoding, and quantization. By compressing memory footprints, parallelizing generation, and reducing numerical precision, these techniques collectively enable the deployment of larger models with longer contexts on existing infrastructure. However, the path forward is not merely incremental; it demands a holistic co-design approach that addresses the dynamic nature of workloads, the heterogeneity of hardware, and the stringent requirements of accuracy and security. As the field evolves, the integration of these methodologies into unified, adaptive serving frameworks will be paramount to unlocking the full potential of large language models in real-world applications.

References

[1] Zixuan Zhou, Xuefei Ning, Ke Hong. (2024). A Survey on Efficient Inference for Large Language Models. arXiv:2404.14294.

[2] Ranran Zhen, Juntao Li, Yixin Ji. (2025). Taming the Titans: A Survey of Efficient LLM Inference Serving. arXiv:2504.19720.

[3] Yao Fu. (2024). Challenges in Deploying Long-Context Transformers: A Theoretical Peak Performance Analysis. arXiv:2405.08944.

[4] Xinyue Ma, Heelim Hong, Taegeon Um. (2026). OrbitFlow: SLO-Aware Long-Context LLM Serving with Fine-Grained KV Cache Reconfiguration. arXiv:2601.10729.

[5] Amey Agrawal, Haoran Qiu, Junda Chen. (2024). Medha: Efficient LLM Inference on Multi-Million Context Lengths Without Approximation. arXiv:2409.17264.

[6] Minsu Kim, Seongmin Hong, RyeoWook Ko. (2025). Oaken: Fast and Efficient LLM Serving with Online-Offline Hybrid KV Cache Quantization. arXiv:2503.18599.

[7] Jianian Zhu, Hang Wu, Haojie Wang. (2024). FastCache: Optimizing Multimodal LLM Serving through Lightweight KV-Cache Compression Framework. arXiv:2503.08461.

[8] Joseph Kampeas, Emir Haleva. (2025). Joint Encoding of KV-Cache Blocks for Scalable LLM Serving. arXiv:2601.03067.

[9] Charlie Chen, Sebastian Borgeaud, Geoffrey Irving. (2023). Accelerating Large Language Model Decoding with Speculative Sampling. arXiv:2302.01318.

[10] Yaniv Leviathan, Matan Kalman, Yossi Matias. (2023). Fast Inference from Transformers via Speculative Decoding. arXiv:2211.17192.

[11] Heming Xia, Zhe Yang, Qingxiu Dong. (2024). Unlocking Efficiency in Large Language Model Inference: A Comprehensive Survey of Speculative Decoding. arXiv:2401.07851.

[12] Dingyu Yao, Chenxu Yang, Zhengyang Tong. (2025). VecInfer: Efficient LLM Inference with Low-Bit KV Cache via Outlier-Suppressed Vector Quantization. arXiv:2510.06175.

[13] Hao Kang, Qingru Zhang, Souvik Kundu. (2024). GEAR: An Efficient KV Cache Compression Recipe for Near-Lossless Generative Inference of LLM. arXiv:2403.05527.

[14] Xing Li, Zeyu Xing, Yiming Li. (2024). KVTuner: Sensitivity-Aware Layer-Wise Mixed-Precision KV Cache Quantization for Efficient and Nearly Lossless LLM Inference. arXiv:2502.04420.

[15] Ranajoy Sadhukhan, Jian Chen, Zhuoming Chen. (2024). MAGICDEC: BREAKING THE LATENCY-THROUGHPUT TRADEOFF FOR LONG CONTEXT GENERATION WITH SPECULATIVE DECODING. arXiv:2408.11049.

[16] Shang Yang, Junxian Guo, Haotian Tang. (2025). LSERVE: EFFICIENT LONG-SEQUENCE LLM SERVING WITH UNIFIED SPARSE ATTENTION. arXiv:2502.14866.

[17] Zhaoyuan Su, Tingfeng Lan, Zirui Wang. (2024). Efficient and Workload-Aware LLM Serving via Runtime Layer Swapping and KV Cache Resizing. arXiv:2506.02006.

[18] Konrad Staniszewski, Adrian Łańcucki. (2025). KV Cache Transform Coding for Compact Storage in LLM Inference. arXiv:2511.01815.

[19] Harshil Vejendla. (2024). H1B-KV: Hybrid One-Bit Caches for Memory-Efficient Large Language Model Inference. arXiv:2510.05529.

[20] Tianyi Zhang, Jonah Yi, Zhaozhuo Xu. (2024). KV Cache is 1 Bit Per Channel: Efficient Large Language Model Inference with Coupled Quantization. arXiv:2405.03917.

[21] Yi Xiong, Hao Wu, Changxu Shao. (2024). LayerKV: Optimizing Large Language Model Serving with Layer-wise KV Cache Management. arXiv:2410.00428.

[22] Shi Qiu, Yifan Hu, Xintao Wang. (2024). Tutti: Making SSD-Backed KV Cache Practical for Long-Context LLM Serving. arXiv:2605.03375.

[23] Zhiqiang Xie, Ziyi Xu, Mark Zhao. (2024). Strata: Hierarchical Context Caching for Long Context Language Model Serving. arXiv:2508.18572.

[24] Cunchen Hu, Heyang Huang, Junhao Hu. (2024). MemServe: Flexible Mem Pool for Building Disaggregated LLM Serving with Caching. arXiv:2406.17565.

[25] Ao Shen, Zhiyao Li, Mingyu Gao. (2024). FastSwitch: Optimizing Context Switching Efficiency in Fairness-Aware Large Language Model Serving. arXiv:2411.18424.

[26] Xingyu Qu, Tianhao Lin, Yiqi Li. (2025). PRISM: Fast Online LLM Serving via Scheduling-Memory Co-Design. arXiv:2605.08581.

[27] Jiayi Yao, Hanchen Li, Yuhan Liu. (2025). CacheBlend: Fast Large Language Model Serving for RAG with Cached Knowledge Fusion. arXiv:2405.16444.

[28] Quqing Zhang, Kai Chen, Ning Liao. (2024). SparseX: Efficient Segment-Level KV Cache Sharing for Interleaved LLM Serving. arXiv:2606.01751.

[29] Zhuohang Bian, Feiyang Wu, Teng Ma. (2024). Tokencake: A KV-Cache-centric Serving Framework for LLM-based Multi-Agent Applications. arXiv:2510.18586.

[30] Zhuohang Bian, Feiyang Wu, Chengrui Zhang. (2026). TokenDance: Scaling Multi-Agent LLM Serving via Collective KV Cache Sharing. arXiv:2604.03143.

[31] Shao Wang, Rui Ren, Lin Gui. (2024). ForkKV: Scaling Multi-LoRA Agent Serving via Copy-on-Write Disaggregated KV Cache. arXiv:2604.06370.

[32] Yuhan Liu, Yuyang Huang, Jiayi Yao. (2024). DroidSpeak: KV Cache Sharing for Cross-LLM Communication and Multi-LLM Serving. arXiv:2411.02820.

[33] Hui Zeng, Daming Zhao, Pengfei Yang. (2025). Lethe: Layer- and Time-Adaptive KV Cache Pruning for Reasoning-Intensive LLM Serving. arXiv:2511.06029.

[34] Hyun-rae Jo, Dongkun Shin. (2024). A2SF: Accumulative Attention Scoring with Forgetting Factor for Token Pruning in Transformer Decoder. arXiv:2407.20485.

[35] Muhammad Adnan, Akhil Arunkumar, Gaurav Jain. (2024). KEYFORMER: KV CACHE REDUCTION THROUGH KEY TOKENS SELECTION FOR EFFICIENT GENERATIVE INFERENCE. arXiv:2403.09054.

[36] Qihui Zhou, Peiqi Yin, Pengfei Zuo. (2024). Progressive Sparse Attention: Algorithm and System Co-design for Efficient Attention in LLM Serving. arXiv:2503.00392.

[37] Lijie Yang, Zhihao Zhang, Zhuofu Chen. (2024). TIDALDECODE: FAST AND ACCURATE LLM DECODING WITH POSITION PERSISTENT SPARSE ATTENTION. arXiv:2410.05076.

[38] Gonçalo Duarte, Miguel Couceiro, Marcos V. Treviso. (2024). EntmaxKV: Support-Aware Decoding for Entmax Attention. arXiv:2605.21649.

[39] Yunfei Cheng, Aonan Zhang, Xuanyu Zhang. (2024). Recurrent Drafter for Fast Speculative Decoding in Large Language Models. arXiv:2403.09919.

[40] Ziqian Zeng, Jiahong Yu, Qianshi Pang. (2024). Chimera: A Lossless Decoding Method for Accelerating Large Language Models Inference by Fusing all Tokens. arXiv:2402.15758.

[41] Siqi Wang, Hailong Yang, Xuezhu Wang. (2024). Minions: Accelerating Large Language Model Inference with Aggregated Speculative Execution. arXiv:2402.15678.

[42] Fahao Chen, Peng Li, Tom H. Luan. (2024). SPIN: Accelerating Large Language Model Inference with Heterogeneous Speculative Models. arXiv:2503.15921.

[43] Hang Wu, Jianian Zhu, Yinghui Li. (2025). SpecRouter: Adaptive Routing for Multi-Level Speculative Decoding in Large Language Models. arXiv:2505.07680.

[44] Haiduo Huang, Fuwei Yang, Zhenhua Liu. (2025). Jakiro: Boosting Speculative Decoding with Decoupled Multi-Head via MoE. arXiv:2502.06282.

[45] Xupeng Miao, Gabriele Oliaro, Zhihao Zhang. (2024). SpecInfer: Accelerating Large Language Model Serving with Tree-based Speculative Inference and Verification. arXiv:2305.09781.

[46] Jikai Wang, Yi Su, Juntao Li. (2024). OPT-Tree: Speculative Decoding with Adaptive Draft Tree Structure. arXiv:2406.17276.

[47] Sen Yang, Shujian Huang, Xinyu Dai. (2023). Multi-Candidate Speculative Decoding. arXiv:2401.06706.

[48] Yao Zhao, Zhitian Xie, Chen Liang. (2024). Lookahead: An Inference Acceleration Framework for Large Language Model with Lossless Generation Accuracy. arXiv:2312.12728.

[49] Jun Zhang, Jue Wang, Huan Li. (2024). Draft & Verify: Lossless Large Language Model Acceleration via Self-Speculative Decoding. arXiv:2309.08168.

[50] Hanling Yi, Feng Lin, Hongbin Li. (2024). Generation Meets Verification: Accelerating Large Language Model Inference with Smart Parallel Auto-Correct Decoding. arXiv:2402.11809.

[51] Feng Lin, Hanling Yi, Hongbin Li. (2024). BiTA: Bi-Directional Tuning for Lossless Acceleration in Large Language Models. arXiv:2401.12522.

[52] Soongyu Choi, Yuntae Kim, Muyoung Son. (2025). Cassandra: Enabling Reasoning LLMs at Edge via Self-Speculative Decoding. arXiv:2605.26558.

[53] Rui Li, Zhaoning Zhang, Libo Zhang. (2024). Nightjar: Dynamic Adaptive Speculative Decoding for Large Language Models Serving. arXiv:2512.22420.

[54] Jaydip Sen, Subhasis Dasgupta, Hetvi Waghela. (2024). Confidence-Modulated Speculative Decoding for Large Language Models. arXiv:2508.15371.

[55] Kuan-Wei Lu, Ding-Yong Hong, Pangfeng Liu. (2025). AdaSD: Adaptive Speculative Decoding for Efficient Language Model Inference. arXiv:2512.11280.

[56] Sudhanshu Agrawal, Wonseok Jeon, Mingu Lee. (2024). AdaEDL: Early Draft Stopping for Speculative Decoding of Large Language Models via an Entropy-based Lower Bound on Token Acceptance Probability. arXiv:2410.18351.

[57] Kaiyu Huang, Hao Wu, Zhubo Shi. (2024). SpecServe: Efficient and SLO-Aware Large Language Model Serving with Adaptive Speculative Decoding. arXiv:2503.05096.

[58] Luxi Lin, Zhihang Lin, Zhanpeng Zeng. (2024). Speculative Decoding Reimagined for Multimodal Large Language Models. arXiv:2505.14260.

[59] Libo Zhang, Zhaoning Zhang, Wangyang Hong. (2025). Sparrow: Text-Anchored Window Attention with Visual-Semantic Glimpsing for Speculative Decoding in Video LLMs. arXiv:2602.15318.

[60] Zhihai Wang, Jie Wang, Jilai Pan. (2024). Accelerating Large Language Model Reasoning via Speculative Search. arXiv:2505.02865.

[61] Edward Kim. (2024). Nevermind: Instruction Override and Moderation in Large Language Models. arXiv:2402.03303.

[62] Jiayi Pan, Chengcan Wang, Kaifu Zheng. (2024). SmoothQuant+: Accurate and Efficient 4-bit Post-Training Weight Quantization for LLM. arXiv:2312.03788.

[63] Yilong Zhao, Chien-Yu Lin, Kan Zhu. (2024). ATOM: LOW-BIT QUANTIZATION FOR EFFICIENT AND ACCURATE LLM SERVING. arXiv:2310.19102.

[64] Hao Zhang, Aining Jia, Weifeng Bu. (2025). FlexQ: Efficient Post-training INT6 Quantization for LLM Serving via Algorithm-System Co-Design. arXiv:2508.04405.

[65] Yeonhong Park, Jake Hyun, SangLyul Cho. (2024). Any-Precision LLM: Low-Cost Deployment of Multiple, Different-Sized LLMs. arXiv:2402.10517.

[66] Hao Gu, Lujun Li, Zheyu Wang. (2024). BTC-LLM: Efficient Sub-1-Bit LLM Quantization via Learnable Transformation and Binary Codebook. arXiv:2506.12040.

[67] Chenyang Yin, Zhenyu Bai, Pranav Venkatram. (2024). TerEffic: Highly Efficient Ternary LLM Inference on FPGA. arXiv:2502.16473.

[68] Wonsuk Jang, Thierry Tambe. (2024). BlockDialect: Block-wise Fine-grained Mixed Format Quantization for Energy-Efficient LLM Inference. arXiv:2501.01144.

[69] Wei Tao, Bin Zhang, Xiaoyang Qu. (2024). Cocktail: Chunk-Adaptive Mixed-Precision Quantization for Long-Context LLM Inference. arXiv:2503.23294.

[70] Wei Tao, Haocheng Lu, Xiaoyang Qu. (2025). MoQAE: Mixed-Precision Quantization for Long-Context LLM Inference via Mixture of Quantization-Aware Experts. arXiv:2506.07533.

[71] Yu Zhang, Mingzi Wang, Lancheng Zou. (2024). MixPE: Quantization and Hardware Co-design for Efficient LLM Inference. arXiv:2411.16158.

[72] Zifan He, Shengyu Ye, Rui Ma. (2026). LUT-LLM: Efficient Large Language Model Inference with Memory-based Computations on FPGAs. arXiv:2511.06174.

[73] Guang Huang, Zeyi Wen. (2025). Quasar: Quantized Self-Speculative Acceleration for Rapid Inference via Memory-Efficient Verification. arXiv:2603.01399.

[74] Bowen Duan, Cong Guo, Chiyue Wei. (2024). EVA: Accelerating LLM Decoding via an Efficient Vector Quantization Architecture. arXiv:2605.24144.

[75] Foteini Strati, Sara Mcallister, Amar Phanishayee. (2024). Dej´ aVu: KV-cache Streaming for Fast, Fault-tolerant Generative LLM Serving. arXiv:2403.01876.

[76] Yiyuan He, Minxian Xu, Jingfeng Wu. (2024). BanaServe: Unified KV Cache and Dynamic Module Migration for Balancing Disaggregated LLM Serving in AI Infrastructure. arXiv:2510.13223.

[77] Jun Wang, Yunxiang Yao, Wenwei Kuang. (2025). OMNIINFER: SYSTEM-WIDE ACCELERATION TECHNIQUES FOR OPTIMIZING LLM SERVING THROUGHPUT AND LATENCY. arXiv:2511.22481.

[78] Jianru Ding, Ryien Hosseini, Pouya Mahdi Gholami. (2026). Observation, Not Prediction: Conversation-Level Disaggregated Scheduling for Agentic Serving. arXiv:2606.01839.

[79] Amey Agrawal, Nitin Kedia, Ashish Panwar. (2024). Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve. arXiv:2403.02310.

[80] Youpeng Zhao, Jun Wang. (2024). ALISE: Accelerating Large Language Model Serving with Speculative Scheduling. arXiv:2410.23537.

[81] Meixuan Wang, Yinyu Ye, Zijie Zhou. (2025). LLM Serving Optimization with Variable Prefill and Decode Lengths. arXiv:2508.06133.

[82] Junyi Chen, Chuheng Du, Renyuan Liu. (2026). TokenFlow: Responsive LLM Text Streaming Serving under Request Burst via Preemptive Scheduling. arXiv:2510.02758.

[83] Alexander Borzunov, Max Ryabinin, Artem Chumachenko. (2023). Distributed Inference and Fine-tuning of Large Language Models Over The Internet. arXiv:2312.08361.

[84] Yao Fu, Leyang Xue, Yeqi Huang. (2024). ServerlessLLM: Low-Latency Serverless Inference for Large Language Models. arXiv:2401.14351.

[85] Fei Fang, Yifan Hua, Shengze Wang. (2024). GenTorrent: Scaling Large Language Model Serving with An Overlay Network. arXiv:2504.20101.

[86] Dongha Yoon, Younghoon Min, Hoshik Kim. (2024). TraCT: Disaggregated LLM Serving with CXL Shared Memory KV Cache at Rack-Scale. arXiv:2512.18194.

[87] Erik Johannes Husom, Arda Goknil, Merve Astekin. (2025). Sustainable LLM Inference for Edge AI: Evaluating Quantized LLMs for Energy Efficiency, Output Accuracy, and Inference Latency. arXiv:2504.03360.

[88] Tollef Emil Jørgensen. (2025). Resource-Efficient Language Models: Quantization for Fast and Accessible Inference. arXiv:2505.08620.

[89] Xing Hu, Yuan Cheng, Dawei Yang. (2024). I-LLM: Efficient Integer-Only Inference for Fully-Quantized Low-Bit Large Language Models. arXiv:2405.17849.

[90] Jianyi Cheng, Cheng Zhang, Zhewen Yu. (2024). A Dataflow Compiler for Efficient LLM Inference using Custom Microscaling Formats. arXiv:2307.15517.

[91] Jiale Xu, Rui Zhang, Cong Guo. (2024). vTensor: Flexible Virtual Tensor Management for Efficient LLM Serving. arXiv:2407.15309.

[92] Jiankun Wei, Abdulrahman Abdulrazzag, Tianchen Zhang. (2024). Privacy Risks of Speculative Decoding in Large Language Models. arXiv:2411.01076.

[93] Panagiotis Georgios Pennas, Konstantinos Papaioannou, Marco Guarnieri. (2024). CacheSolidarity: Preventing Prefix Caching Side Channels in Multi-tenant LLM Serving Systems. arXiv:2603.10726.

[94] Michael Cunningham. (2026). Privacy-Aware Split Inference with Speculative Decoding for Large Language Models over Wide-Area Networks. arXiv:2602.16760.

[95] Xupeng Miao, Gabriele Oliaro, Zhihao Zhang. (2025). Towards Efficient Generative Large Language Model Serving: A Survey from Algorithms to Systems. arXiv:2312.15234.

[96] Joris Köster, Zixuan Liu, Siavash Khajavi. (2025). MemBoost: A Memory-Boosted Framework for Cost-Aware LLM Inference. arXiv:2603.26557.

[97] Wonung Kim, Yubin Lee, Yoonsung Kim. (2025). Pimba: A Processing-in-Memory Acceleration for Post-Transformer Large Language Model Serving. arXiv:2507.10178.

[98] Aleksandar Botev, Soham De, Samuel L Smith. (2024). RecurrentGemma: Moving Past Transformers for Efficient Open Language Models. arXiv:2404.07839.

[99] Jared Fernandez, Clara Na, Vashisth Tiwari. (2025). Energy Considerations of Large Language Model Inference and Efficiency Optimizations. arXiv:2504.17674.

[100] Soham Poddar, Paramita Koley, Janardan Misra. (2024). Towards Sustainable NLP: Insights from Benchmarking Inference Energy in Large Language Models. arXiv:2502.05610.